World Class Scholars Digital Governance Architecture

Purpose

This document describes the digital governance architecture for the World Class Scholars (WCS) platform, covering websites, iOS applications, backend services, and R&D evidence management. The goal is to ensure founder-level control, secure delegation, and auditable evidence for research and development (R&D) and grant reporting obligations.

The governance model is implemented as code within the primary WCS web application (Next.js with App Router), backed by a Supabase/Postgres backend using role-based access control (RBAC) and Row Level Security (RLS).

High-level architecture

- **Client**: Next.js App Router application deployed on Vercel, serving both public website pages and an internal governance console.
- **Backend**: Supabase (Postgres + Auth) providing authentication, authorization, storage, and RLS-enforced data access.
- **Mobile**: iOS apps (SwiftUI) that authenticate via Supabase and consume the same RBAC-controlled APIs as the web console.
- **Source control**: GitHub repositories for code, migrations, governance documentation, and generated evidence reports.

The governance console is implemented under protected routes in the existing WCS web app (e.g. `/dashboard`, `/access`, `/rd-projects`, `/grants`, `/audit`), accessible only to authenticated staff with appropriate roles and scopes.

Repository map

Component	Repository
Governance console + migrations	`chrsappiah-cloud/wcs-governance`
Public marketing site (current)	`chrsappiah-cloud/wcs-full`
iOS applications	Separate repos; shared Supabase project

Ownership model

The founder retains ultimate ownership of all critical digital assets:

- Domain registration and DNS for WCS web properties.
- Supabase project ownership (including database, auth, and storage).
- Vercel project and team ownership for the WCS web application.
- GitHub organisation ownership for WCS source code repositories.
- Apple Developer Program and App Store Connect Account Holder status for all WCS iOS applications.

Operational staff receive restricted permissions within this framework through the governance console and scoped roles in Supabase; they do not hold independent ownership of these assets.

Role-based access control (RBAC)

RBAC is implemented in Supabase using:

- **`roles`**: named roles such as `founder_admin`, `platform_admin`, `product_manager`, `content_manager`, `ios_release_manager`, `rd_coordinator`, `finance_grants_manager`, `support_lead`, `engineer`, and `contractor`.
- **`permissions`**: atomic capabilities such as `manage_access`, `publish_content`, `manage_release`, `create_rd_record`, `approve_rd_record`, `export_evidence_pack`, and `view_audit_logs`.
- **`resource_scopes`**: context identifiers such as `org-global`, `website-main`, `etherealveil-ios`, `wcs-platform-rd-2026`, and `grant-rd-funding-2026`.
- **`user_role_assignments`**: mappings that assign a role to a user for a specific scope.

Helper functions (`user_has_permission_for_scope`, `is_founder_admin`) and RLS policies enforce access decisions in the database, rather than relying only on application code.

This structure supports fine-grained delegation (for example, allowing a content manager to publish only on `website-main`, while an R&D coordinator can create evidence only for defined R&D projects) and centralizes control decisions for audits.

See roles-and-scopes.md for the full role matrix.

Governance console

The governance console is implemented as a set of protected routes in the existing WCS Next.js app using the App Router.

Key modules:

- **Dashboard**: Overview of governance status across access, R&D projects, and grants.
- **Access**: Management of role assignments and scopes for staff and contractors, restricted to the founder and access administrators.

- **R&D Projects**: Registration of R&D projects, including technical uncertainties, objectives, and status.
- **R&D Evidence**: Logging of hypotheses, experiments, results, cost links, and decisions as structured evidence records.
- **Grants**: Recording of grant milestones, expenditure mapping, and reporting deadlines.
- **Audit**: Viewing of audit logs showing who changed what, when, and under which role.

The console uses Supabase Auth with server-side session handling and protects routes via server-based checks before rendering, as recommended for Next.js App Router applications.

Authentication and authorization

Authentication is handled by Supabase Auth, with the following characteristics:

- Users sign in to the governance console via Supabase email-based authentication.
- Sessions are stored and read server-side using the `@supabase/ssr` package and Next.js cookies.
- A minimal set of claims (such as `org_role` and `is_staff`) is added via a custom access token hook; fine-grained access remains in the database.

Authorization is enforced through:

- Server-side guards (`requireStaff`, `requirePermission`) that check user roles and scopes via Supabase RPC calls.
- Row Level Security (RLS) policies that constrain what rows can be selected, inserted, updated, or deleted, based on helper functions and the authenticated user ID.

This layered approach ensures that governance rules are enforced both in the Next.js server code and at the database level, reducing reliance on front-end checks alone.

Enforcement layers (implementation)

1. Sign-in → JWT claims (`is_staff`, `org_role`, `founder_access`)
2. `requireStaff()` — coarse console gate
3. `requirePermission(key, scope)` — RPC permission check + founder override
4. RLS — Postgres policies on sensitive tables
5. Audit triggers — automatic JSON snapshots on mutations

Audit and logging

The system maintains an auditable history of governance actions by:

- Using Supabase Auth audit logs for authentication-related events, such as sign-ins and account changes.
- Storing application-level logs in a dedicated `audit_logs` table, populated by Postgres triggers on sensitive tables such as `user_role_assignments`, `approval_requests`, and `rd_evidence_records`.

Each audit entry records:

- The acting user (`actor_user_id`).
- The affected table (`entity_type`) and row (`entity_id`).
- The operation (`insert`, `update`, `delete`).
- The `before_state` and `after_state` as JSON snapshots.
- A timestamp.

This audit trail supports accountability for internal governance and provides evidence for external reviewers that controls operate as described.

R&D evidence governance

R&D governance is integrated into the normal development workflow via:

- **`rd_projects`**: defining each R&D project, technical uncertainty, and objective in alignment with Australian R&D tax guidance on eligible activities.
- **`rd_evidence_records`**: capturing contemporaneous evidence entries (hypotheses, experiments, results, cost links, decisions, and artifacts) with links to commits, builds, and cost references.
- **Monthly and milestone evidence packs** generated from these records and stored as Markdown files in the repository under `docs/rd-evidence`.

Evidence records are created by engineers and R&D coordinators through the console using server actions that enforce scope-based permissions and record attribution in the database. This ensures that R&D evidence is systematic, contemporaneous, and linked to underlying technical and financial records, as recommended by Australian R&D record-keeping guidance.

Export pipeline

Method	Output
Server action `createRDEvidence`	Inserts into `rd_evidence_records` (RLS + audit trigger)
Console `/rd-projects/monthly`	Human-readable monthly pack view
`npm run export:rd -- <scope> <year> <month>`	`docs/rd-evidence/<scope>-<year>-<month>.md`
GitHub Actions `.github/workflows/rd-report.yml`	Automated export + commit to Git

Grant and funding reporting

Grant-related governance includes:

- Tracking of grant scopes (for example, `grant-rd-funding-2026`) in `resource_scopes`.
- Recording of grant milestones, expenditure mapping, and acquittal status in the governance console.
- Generation of periodic and final evidence packs using the same R&D evidence data and structured reporting views.

This supports compliance with government grant reporting requirements, which typically require progress updates, final reports, financial reconciliation, and evidence that expenditure was used as approved.

Documentation and report generation

Documentation and reports are integrated with the WCS GitHub repos as follows:

- Governance SQL migrations and Next.js console code are stored and versioned in the main WCS repository under `supabase/migrations` and `app/(console)` respectively.
- R&D evidence packs are generated from Supabase data using a Node.js/TypeScript script that queries the database and writes Markdown files into `docs/rd-evidence`, which are then committed to Git.
- GitHub Actions workflows automate periodic evidence export, producing CI artifacts and commits with updated documentation.
- **PDF exports** of governance architecture and R&D evidence packs are generated via `npm run export:pdf` and stored under `docs/governance/exports/` and `docs/rd-evidence/` for acquittal attachments and external review.

This approach ensures that governance configuration, operational logs, and R&D evidence are treated as code and documents within source control, improving reproducibility and audit readiness.

Schema migrations

Applied in order via Supabase SQL Editor or CLI (`supabase db push`):

1. `001_core_governance.sql`
2. `002_permissions_seed.sql`
3. `003_rls.sql`
4. `004_audit_triggers.sql`

Summary

The WCS digital governance architecture:

- Centralizes control in a founder-owned Supabase and Next.js platform.
- Uses RBAC and RLS to delegate limited, scope-aware access to staff and contractors.
- Embeds audit logging at both auth and application levels.
- Integrates R&D evidence capture and grant reporting into normal delivery workflows.

- Leverages GitHub for standardised documentation and report generation.

Together, these elements create a governance system that supports operational needs while providing defensible, auditable evidence for R&D tax incentives and government grant reporting.

Related documentation

- Roles and scopes
- R&D evidence exports
- Developer prompts
- Integration with wcs-full
- iOS client integration